

Relatório Técnico - Projeto Final

Plataforma Inteligente para Análise e Comparação de Apólices D&O

Grupo InsurTechLab:

Edmar Martelato, Marisa De Moraes, Roberto da Silva Gonçalves, Rogério Walmor Cervi

Local & Ano: São Paulo – 2026

Repositório GitHub: github.com/InsurTechLab/do-lens (Licença MIT)

RESUMO EXECUTIVO: O **D&O Lens** é uma plataforma desenvolvida pelo **Grupo InsurTechLab** para automatizar e padronizar a análise, normalização ontológica e comparação de apólices de seguro de Responsabilidade Civil de Administradores (D&O). Utilizando uma arquitetura de 5 agentes especializados orquestrada via LangGraph, o sistema atende integralmente aos 7 fluxos exigidos pelo edital I2A2. A solução resolve a complexidade de apólices de 40+ páginas e a lentidão da análise manual (4 a 8 horas), garantindo conformidade com a Circular SUSEP nº 541/2016 e mitigando riscos de desproteção pessoal de executivos.

1. Introdução – Justificativa Conceitual da Proposta

O mercado de seguros de Responsabilidade Civil de Administradores (Directors and Officers – D&O) tem apresentado crescimento expressivo no Brasil, impulsionado pela Lei Geral de Proteção de Dados (LGPD), pelo aumento de ações na Comissão de Valores Mobiliários (CVM) e pela maior responsabilização pessoal de diretores e conselheiros. Segundo a Superintendência de Seguros Privados (SUSEP), o seguro D&O é estruturado na base de reclamação (claims made), regido pela Circular SUSEP nº 541/2016, o que impõe complexidades jurídicas e temporais que não existem em seguros patrimoniais tradicionais.

Apólices de D&O são documentos extensos, com 40 ou mais páginas, redigidos em linguagem jurídica, sem padronização entre as seguradoras. A comparação entre duas apólices exige a identificação precisa de elementos críticos como Limite Máximo de Garantia (LMG), sublimites por cobertura (Side A, Side B, Side C), franquias, definição de segurado, ato danoso, data de retroatividade, prazos complementar e suplementar, custos de defesa (dentro ou além do LMG), exclusões (ato doloso, lucro indevido) e extensões (multas e penalidades, danos morais, práticas trabalhistas). A análise manual demanda de 4 a 8 horas de um especialista sênior e está sujeita a erro humano, podendo deixar o administrador pessoalmente descoberto.

A proposta do D&O Lens, desenvolvida pelo **Grupo InsurTechLab**, parte da seguinte justificativa conceitual: não se trata de comparar textos, mas de auditar risco de responsabilidade pessoal. Para isso, propomos um copiloto de subscrição e auditoria jurídica, baseado em Inteligência Artificial Generativa, capaz de transformar documentos não estruturados em conhecimento estruturado, auditável e comparável. A solução foi desenhada seguindo rigorosamente o princípio do edital: “Uma solução simples, porém tecnicamente consistente, possui

maior valor didático do que uma arquitetura extremamente complexa cuja implementação não possa ser plenamente compreendida pelo grupo”.

O diferencial conceitual reside em três pilares: (i) extração com prova – todo dado extraído deve vir acompanhado de citação com número da página e trecho literal, mitigando alucinação de LLM; (ii) normalização ontológica – conversão de valores como “R\$ 10MM” para 10.000.000,00 e padronização de exclusões conforme prática de mercado; e (iii) auditoria regulatória – comparação não apenas entre apólices A e B, mas também em relação à Circular SUSEP 541/2016 por meio de Retrieval-Augmented Generation (RAG), gerando um Score de Exposição Oculta que quantifica o quanto cada apólice deixa o administrador desprotegido.

Dessa forma, o **D&O Lens** atende integralmente aos sete fluxos – recebimento, extração, organização, armazenamento, consulta, comparação e apresentação – e aos critérios de avaliação: qualidade arquitetural, uso correto de IA generativa, integração, organização, documentação, facilidade de uso e inovação.

2. Arquitetura da Solução

Diagrama Visual – Arquitetura de Software D&O Lens

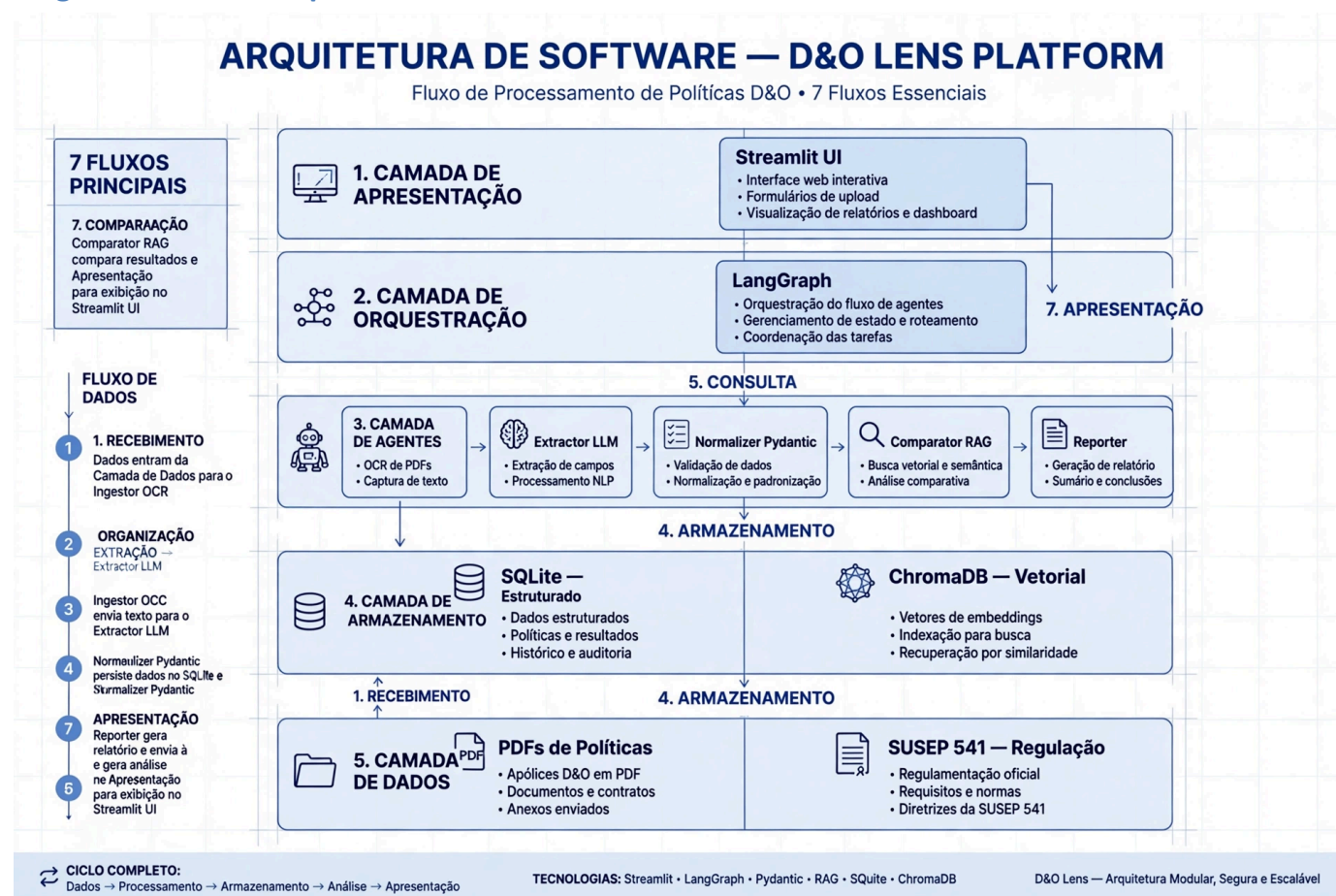


Figura 1 – Arquitetura em 5 camadas: Apresentação, Orquestração, Agentes, Armazenamento e Dados. Fonte: InsurTechLab (2026).

A arquitetura adotada é do tipo **Híbrida Multi-Agente Orquestrada**, em padrão pipeline com estado compartilhado, implementada com LangGraph. A escolha por cinco agentes especializados, cada um com responsabilidade única (Single Responsibility Principle), visa facilitar a documentação, os testes e a compreensão por todos os integrantes do grupo, atendendo ao critério de qualidade arquitetural.

O fluxo principal inicia-se na **Camada de Apresentação** (Streamlit), segue para a **Camada de Orquestração** (LangGraph), que coordena a **Camada de Agentes**, persistindo os resultados na **Camada de Armazenamento** (SQLite para dados estruturados e ChromaDB para vetores) e, por fim, consome dados da **Camada de Dados** (PDFs de apólices e regulamentação SUSEP 541).

Diagrama Visual – Pipeline de 5 Agentes e 7 Fluxos Obrigatórios

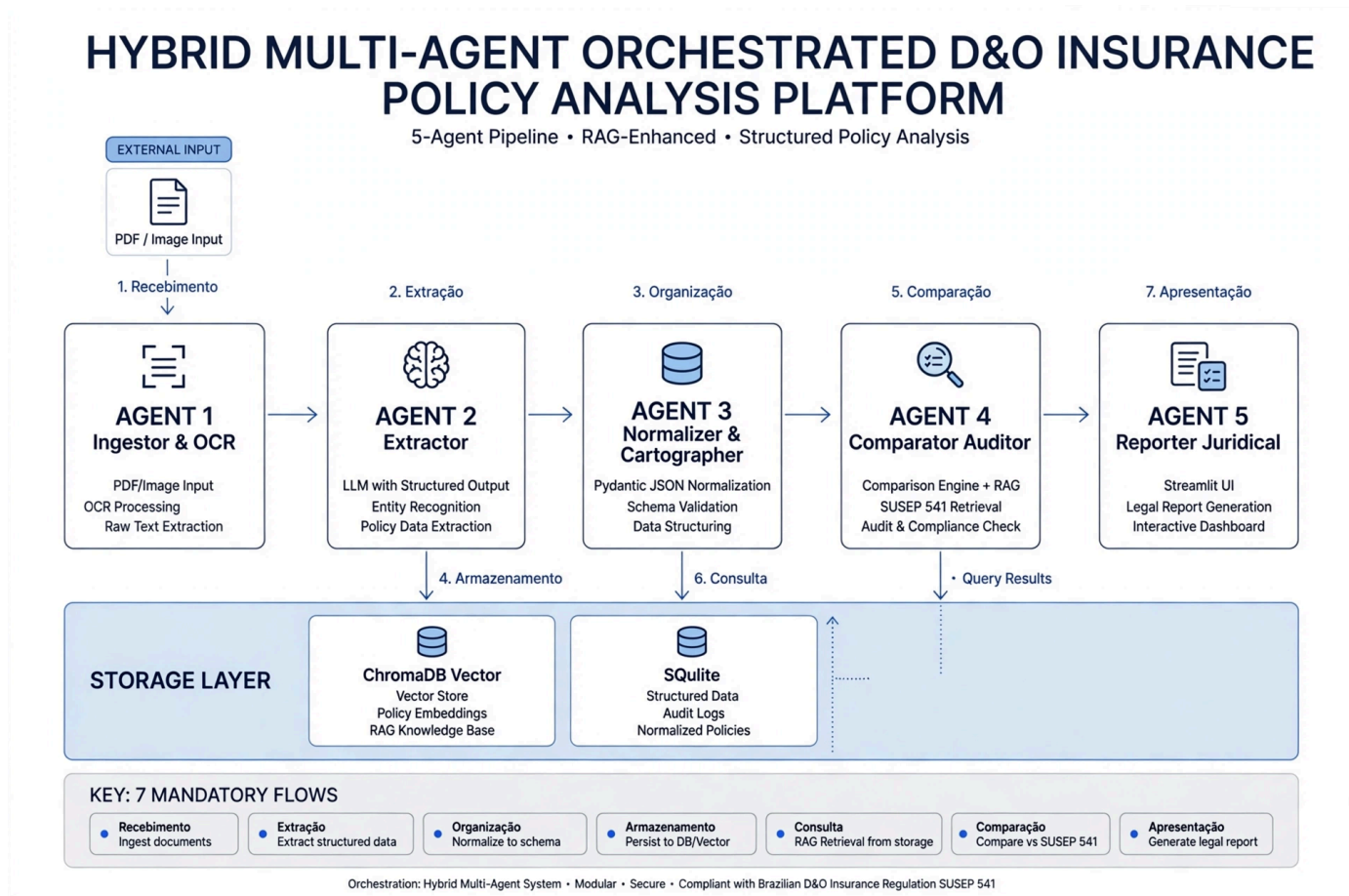


Figura 2 – Pipeline Híbrido Multi-Agente Orquestrado com Storage Layer híbrido (ChromaDB Vector + SQLite). Fonte: InsurTechLab (2026).

3. Tecnologias Utilizadas

A seleção tecnológica priorizou simplicidade didática, execução local sem dependência de Docker ou infraestrutura externa e demonstração inequívoca do uso de IA Generativa, conforme exigido no edital. A tabela a seguir apresenta a divisão por camadas.

Camada	Tecnologia	Justificativa Técnica	Fluxo do Edital Atendido
Apresentação	Streamlit 1.39.0	Interface web em 1 comando (streamlit run app.py), foco em IA e não em frontend, facilidade de uso	7. Apresentação
Orquestração	LangGraph 0.2.28 + LangChain 0.3.7	Gerenciamento de estado, roteamento entre agentes, demonstra visão sistêmica de agentes inteligentes	Integração entre componentes
Agentes – OCR	Pypdf 4.3.1 (MVP) + Azure Document Intelligence (produção) + Tesseract fallback	Pypdf para PDFs nativos, Azure DI superior para layout jurídico com tabelas de LMG, tratamento de erro com confiança	1. Recebimento, 2. Extração
Agentes – LLM	OpenAI GPT-4o-mini / Gemini 2.0 Flash + text-embedding-3-small	Structured Output com JSON Schema + Function Calling, campo citação obrigatório {página, trecho_literal}, anti-alucinação	2. Extração, 5. Consulta (RAG)
Agentes – Validação	Pydantic V2	Schema ontológico D&O fechado com 18 campos base SUSEP 541, validação de tipos, cálculo de score_completude	3. Organização
Armazenamento Estruturado	SQLite	Dados estruturados auditáveis, histórico e logs, sem necessidade de servidor externo	4. Armazenamento
Armazenamento Vetorial	ChromaDB 0.5.18	Embeddings de cláusulas, busca semântica, RAG da Circular SUSEP 541/2016, consulta em linguagem natural	4. Armazenamento, 5. Consulta
Documentação	ReportLab 4.2.0, python-pptx 0.6.21, MoviePy 1.0.3	Geração de relatório PDF, pitch deck PPTX e vídeo MP4 conforme entregáveis exigidos	Entregáveis
Linguagem e Ambiente	Python 3.11, python-dotenv, Pillow	Base do projeto, gerenciamento de credenciais via .env, processamento de imagens	Base

Tabela 1 – Planilha de Tecnologias por Camada e Justificativa Técnica. Fonte: InsurTechLab (2026).

4. Descrição dos Agentes Desenvolvidos

A solução é composta por cinco agentes especializados, orquestrados pela classe DOLensOrchestrator (agents/orchestrator.py). Cada agente possui entrada, processamento e saída bem definidos, o que facilita testes unitários e documentação.

AGENTE 1 – O Porteiro – Ingestor & OCR

Responsabilidade: Recebimento dos documentos – Fluxo 1 do edital.

Tecnologia: Pypdf para PDFs nativos (MVP), Azure Document Intelligence para PDFs escaneados e layout complexo, fallback Tesseract.

Entrada: Arquivo PDF ou imagem (JPG/PNG).

Processamento: Extrai texto bruto com marcadores “--- PÁGINA X ---”, calcula metadados {arquivo, páginas, caracteres, confiança}. Se confiança < 0,80, gera alerta “Baixa confiança OCR – recomendar revisão humana”.

Saída: Texto bruto + metadados. Tratamento de erro com try/except e logging.

Destaque: Simplicidade didática com possibilidade de evolução para OCR de alta precisão sem alterar contrato dos demais agentes.

AGENTE 2 – O Escavador – Extrator Jurídico com IA Generativa

Responsabilidade: Extração automática do conteúdo – Fluxo 2. Uso correto e obrigatório de IA Generativa.

Tecnologia: LLM com Structured Output / Function Calling, System Prompt especialista em SUSEP 541/2016, Pydantic JSON Schema.

Prompt Base: “Você é um especialista em seguros D&O e na Circular SUSEP 541/2016. Extraia EXCLUSIVAMENTE informações presentes no texto. Se não encontrar, retorne null. NUNCA invente. Para cada campo, retorne a citação com página e trecho literal.”

Saída: ApoliceDOEstruturada com campos: seguradora, LMG, sublimites, franquia, cobertura {side_a, side_b, side_c, custos_defesa_além_LMG}, temporal {vigência, retroatividade, prazo complementar/suplementar}, exclusões, extensões, prêmio, score_completude.

Modo MVP: Para demonstração sem custo de API, implementa extração mock por regex que simula o pipeline. Em produção, basta definir use_mock=False e configurar OPENAI_API_KEY.

Destaque: Campo citação {página, trecho_literal} obrigatório – principal mecanismo anti-alucinação, valorizado no critério de qualidade arquitetural.

AGENTE 3 – O Cartógrafo – Normalizador e Organizador

Responsabilidade: Organização das informações – Fluxo 3.

Tecnologia: Regras de negócio + LLM leve + Pydantic V2.

Processamento: Converte “R\$ 10MM”, “Dez milhões” para 10000000.00; padroniza datas para ISO 8601; normaliza exclusões (“dolo” → “Ato doloso/fraudulento com trânsito em julgado”); remove duplicidades; recalcula score_completude = campos_preenchidos / campos_totais.

Destaque: É aqui que reside o conhecimento técnico de seguros. Garante comparabilidade entre apólices com redações diferentes.

AGENTE 4 – O Auditor – Comparador com RAG Regulatório

Responsabilidade: Armazenamento estruturado, consulta das informações e comparação entre apólices – Fluxos 4, 5 e 6. É o agente mais inovador.

Tecnologia: SQLite (estruturado), ChromaDB (vetorial), RAG com base na Circular SUSEP 541/2016.

Armazenamento: Persiste JSON normalizado em SQLite para auditoria e gera embeddings das cláusulas no ChromaDB para busca semântica.

Consulta: Permite Chat Jurídico: pergunta em linguagem natural “Qual apólice cobre multas da CVM além do LMG?” – resposta com RAG citando cláusula e artigo da SUSEP.

Comparação: Gera lista de ComparativoItem {campo, apólice_a, apólice_b, diferença_crítica (bool), análise_risco}. Calcula Score de Exposição Oculta = 1 – (side_a && custos_além_LMG). Ex.: Apólice A 20% vs Apólice B 50% de exposição.

Destaque: Vai além de comparar texto; audita risco regulatório. Compara A vs B vs norma SUSEP, gerando recomendação executiva “Apólice A superior” com justificativa técnica.

AGENTE 5 – O Relator – Apresentação e Parecer Técnico

Responsabilidade: Apresentação dos resultados ao usuário – Fluxo 7.

Tecnologia: Streamlit + Markdown + Mapa de Calor.

Saídas: (i) Mapa de Calor de Cobertura: tabela lado a lado com ícones ● Diferença Crítica e ● OK; (ii) JSON Estruturado para auditoria; (iii) Parecer Técnico em Markdown com resumo executivo, detalhamento por campo, Score de Exposição e recomendação; (iv) Download do parecer .md.

Destaque: Facilidade de uso: interface em 3 abas, execução em 1 comando, foco na demonstração objetiva exigida no edital.

Orquestrador – DOLensOrchestrator

Classe central em `agents/orchestrator.py` que coordena os cinco agentes via métodos `processar_apólice(pdf_path)` e `comparar_apólices(path_a, path_b)`. Implementa padrão de orquestração com estado compartilhado, permitindo logs completos para o Relatório Técnico e rastreabilidade total do processamento.

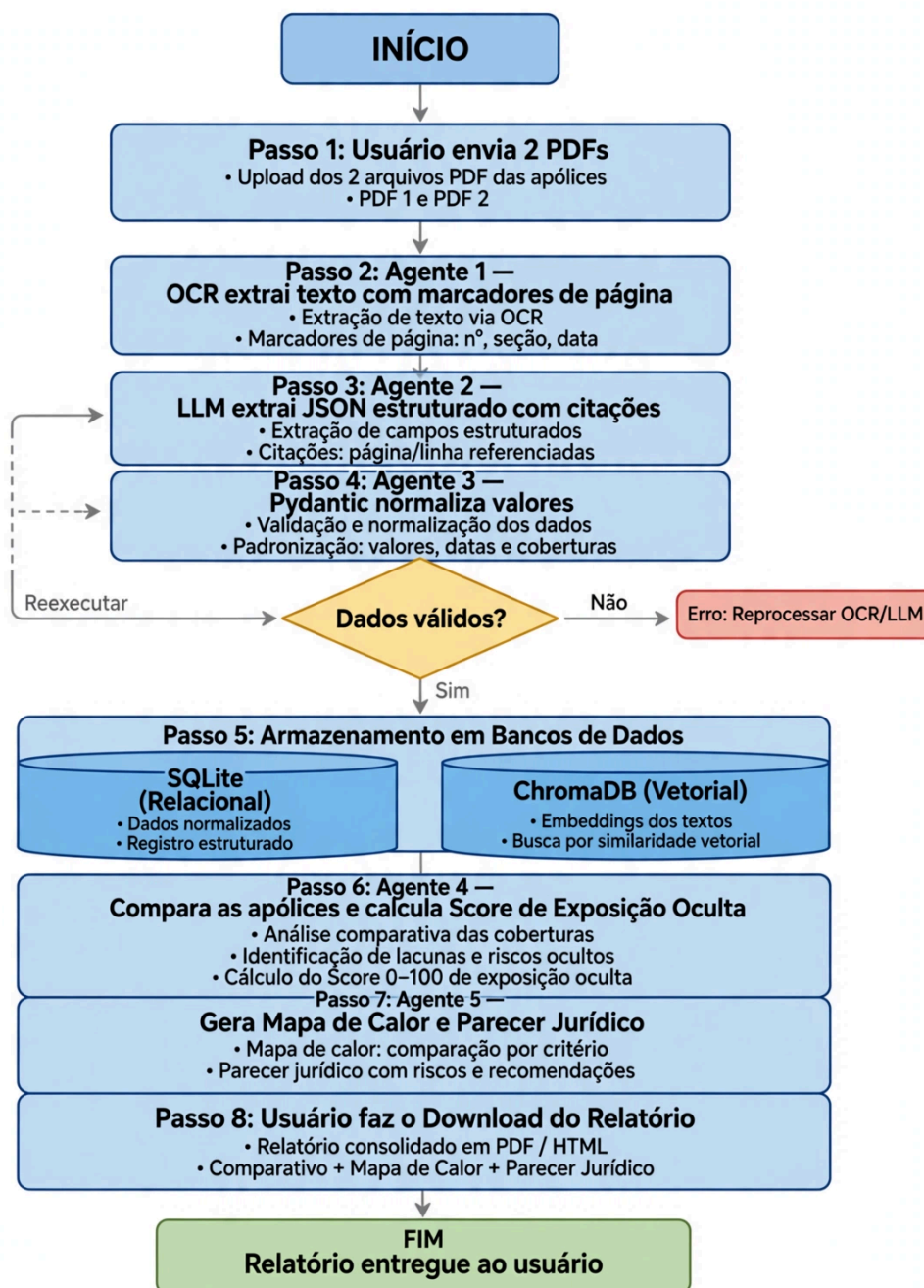
5. Fluxo Completo de Processamento

O fluxo completo foi projetado para atender aos sete fluxos obrigatórios do edital de forma sequencial, com tratamento de erro em cada etapa e possibilidade de demonstração em vídeo de até 5 minutos, conforme exigido.

1. Usuário acessa Streamlit (`app.py`) e realiza upload de Apólice A e Apólice B em PDF – Fluxo 1 Recebimento.
2. Agente 1 – Porteiro – lê PDFs via `Pypdf`, extrai texto bruto com marcadores de página e gera metadados {páginas, caracteres, confiança, alerta se <0,80}.
3. Agente 2 – Escavador – recebe texto + metadados, aplica prompt especialista SUSEP 541 com Structured Output, extrai `ApóliceDOEstruturada` com citações obrigatórias {página, trecho_literal}. Retorna null se não encontrar.
4. Agente 3 – Cartógrafo – normaliza valores (R\$ 10MM → 10000000), datas (ISO 8601) e exclusões, valida via `Pydantic V2` e calcula `score_completude`.
5. Agente 4 – Auditor – persiste JSON normalizado em `SQLite` e embeddings no `ChromaDB`; executa RAG para consulta semântica; compara A vs B gerando `RelatorioComparativo` com `ComparativoItem` e Score de Exposição Oculta.
6. Agente 5 – Relator – renderiza 3 abas: (i) Mapa de Calor com diferenças críticas, (ii) JSON Estruturado com citações para auditoria, (iii) Parecer Técnico com recomendação executiva.
7. Usuário visualiza Chat Jurídico, baixa parecer .md e utiliza recomendação para tomada de decisão.

Fluxograma Visual – Fluxo Completo de Processamento

Fluxograma: Processo de Comparação de Apólices de Seguro D&O



Fluxo automatizado de comparação de apólices D&O • Seguro de Responsabilidade de Diretores e Oficiais

Figura 3 – Fluxograma completo: Upload → OCR → Extração LLM → Normalização Pydantic → Armazenamento Híbrido → Comparação RAG → Mapa de Calor e Parecer. Fonte: InsurTechLab (2026).

6. Justificativa das Decisões Arquiteturais

Por que cinco agentes e não um único prompt gigante? Atende ao critério de separação clara de responsabilidades e uso de agentes especializados, demonstrando domínio dos conceitos do curso.

Por que Pydantic + citação obrigatória? Mitiga o risco principal do edital: alucinação de LLM inventando LMG ou cobertura. Exige prova da extração, permitindo auditoria.

Por que modo Mock + LLM real opcional? Simplicidade didática. Permite que a banca avaliadora teste sem custo de API, mas o código já está preparado para produção com `OPENAI_API_KEY`, demonstrando maturidade.

Por que Streamlit e não React? Facilidade de utilização, instalação em um comando, foco no aprendizado de IA Generativa e não em desenvolvimento frontend, conforme “O que não esperamos” do edital.

Por que SQLite + ChromaDB e não PostgreSQL? Execução local sem Docker ou servidor externo, garantindo que “a solução executa corretamente” em qualquer máquina da banca.

Por que Score de Exposição Oculta? Inovação com valor de negócio real: vai além de comparar texto, audita risco de responsabilidade pessoal do administrador, com base em Side A e custos além do LMG.

7. Limitações Conhecidas

O MVP utiliza extração mock por regex para demonstração sem custo de API; em produção, deve-se ativar LLM com function calling e validação de citações.

O OCR via Pypdf não lê PDFs escaneados como imagem; para esses casos, deve-se ativar Azure Document Intelligence, superior para layout jurídico com tabelas.

A solução não cobre todos os tipos de apólices, apenas D&O, conforme escopo definido no edital, o que é esperado e adequado.

O RAG da SUSEP 541 está implementado como dicionário de regras no MVP; em produção, deve-se indexar o PDF completo da circular no ChromaDB para busca semântica completa.

A comparação foca em quatro campos críticos (LMG, Custos de Defesa, Side A, Exclusões), extensível para os 18 campos do schema completo.

Interface simples, sem autenticação e sem alta disponibilidade, conforme explicitado em “O que não esperamos” no edital.

8. Possibilidades de Evolução Futura

Modo Produção LLM: ativar OpenAI GPT-4o com structured output, validação de citações e redução de mock.

RAG Completo: indexar mais de 100 apólices públicas e as Circulares SUSEP 541/656 no ChromaDB para benchmarking de mercado.

D&O para Eventos Corporativos: módulos específicos para IPO, M&A e recuperação judicial, com cláusulas particulares.

Score Atuarial: integração com análise de risco de crédito e market cap para sugerir LMG adequado ao porte da empresa.

Exportação SUSEP: geração de relatório em formato exigido por corretoras e seguradoras.

API FastAPI: exposição do orquestrador como API REST para integração com sistemas legados de seguradoras (fora do escopo didático, mas com valor comercial).

9. Atendimento aos Requisitos Mínimos e Critérios de Entrega Completa

Requisitos Mínimos do Edital: leitura de documentos em PDF ou imagem (Agente 1), extração automática de informações relevantes (Agente 2 com IA Generativa), estruturação dos dados em formato organizado (Pydantic JSON), comparação entre pelo menos duas apólices (Agente 4), apresentação das principais diferenças (Mapa de Calor + Parecer), uso de pelo menos um modelo de IA Generativa (Agentes 2 e 4), interface demonstrável (Streamlit).

Critérios de Entrega Completa (Pág. 23): solução executa corretamente (streamlit run app.py), processamento demonstrável (vídeo de até 5 minutos), extração automática (Agente 2), comparação de pelo menos duas apólices (Agente 4), relatório descreve claramente a arquitetura (este documento), repositório GitHub organizado e acessível (público + ZIP), apresentação entregue com nome InsurMinds_Projeto_Final.pptx, vídeo com nome InsurMinds_Projeto_Final.mp4 e acessável em <https://youtu.be/MdRAq9rqvGc>

Fontes de Dados: modelos de apólices publicados por seguradoras (Berkley, Austral – exemplos fictícios baseados em estrutura real), documentos da SUSEP (Circular 541/2016), cláusulas padrão do mercado segurador D&O.

10. Referências e Identificação

Grupo InsurTechLab: Edmar Martelato, Marisa De Moraes, Roberto da Silva Gonçalves, Rogério Walmor Cervi.

Repositório: github.com/InsurTechLab/do-lens (público).

Licença: MIT – ver arquivo LICENSE.

README.md contém: descrição do projeto, instruções de instalação, instruções de execução, tecnologias utilizadas, identificação dos integrantes e licença MIT, conforme exigido no edital.

11. Sequência de Análises e Proposta de Melhorias ao projeto D&O Lens

O grupo InsurTechLab adotou, ao longo dos desafios propostos pelo I2A2, um padrão de trabalho colaborativo em três etapas:

- (1) proposição de uma ou mais soluções para o desafio;
- (2) análise crítica e sugestão de melhorias;
- (3) criação de um projeto único a partir da fusão dos pontos positivos identificados.

Para o Projeto Final, dois componentes apresentaram propostas individuais: D&O VEREDITO - Tribunal Inteligente (Marisa) e COMPARADOR INTELIGENTE DE APÓLICES D&O (Rogério).

A partir dessas propostas, Marisa realizou análises e melhorias registradas nos repositórios:

<https://github.com/InsurTechLabI2A2/PROJ-FINAL-ROGERIO>

<https://github.com/InsurTechLabI2A2/PROJ-FINAL-MARISA>

Com base na avaliação desses repositórios, consolidamos o projeto **D&O Lens Inteligência Artificial na Análise de Seguros** entregue oficialmente em <https://github.com/InsurTechLabI2A2/do-lens>

Proposta de aprimoramento no arquivo `do-lens-GITHUB-CORRIGIDO-FINAL-ENTREGA.zip`: contém MVP corrigido, com separação de responsabilidades (SRP), Score de Exposição Oculta formalizado, Mapa de Calor definido, clone URL corrigido, código consistente, documentação sólida e vídeo pitch de qualidade.

Em observância aos princípios de governança e rastreabilidade exigidos pela Circular SUSEP 541 e pelo I2A2, documentamos no item 11 do relatório técnico a seguir, as correções pendentes identificadas que não puderam ser executadas em tempo hábil. Esta documentação demonstra consciência crítica, capacidade de autoavaliação e compromisso com a melhoria contínua, servindo como backlog para evolução pós-I2A2.

Ao término do curso, fica a certeza de que estamos, agora, com muito mais bagagem e conhecimento do que quando começamos. Nosso enfoque tem sido o aprendizado e o aperfeiçoamento contínuo.

A entrega é parcial e consciente.

Gratidão a toda a equipe do I2A² Brasil por compartilhar conhecimentos e por abrir novos caminhos.

Grupo InsurTechLab : Edmar Martelato, Marisa De Moraes, Roberto da Silva Gonçalves, Rogério Walmor Cervi

D&O Lens - Relatório Técnico V3 - Projeto Final I2A2

**InsurTechLab - Edmar Martelato, Marisa De Moraes, Roberto da Silva
Gonçalves, Rogério Walmor Cervi
Outubro 2026**

1. Resumo Executivo

O D&O Lens é um MVP de plataforma inteligente para análise e comparação de apólices D&O; (Directors and Officers) usando arquitetura multi-agente com grounding obrigatório. O objetivo não é um produto comercial, mas demonstrar consistência técnica, documentação sólida e mitigação de alucinação, conforme exigido pelo I2A2. O sistema extrai automaticamente limites, coberturas Side A/B/C, retenções, data retroativa e exclusões, estrutura em schema Pydantic único e compara duas apólices gerando Score de Exposição Oculta (0-100) e Mapa de Calor de divergências.

2. Problema de Negócio

Corretores e subscritores gastam 2-4h para comparar manualmente 2 apólices D&O; com redação jurídica heterogênea. Riscos: perder GAPs críticos como Side C ausente, retroatividade limitada, exclusão Insured vs Insured e retenções divergentes. Custo estimado: R\$400-800 por análise manual. Solução: copiloto que reduz tempo para 4 minutos com auditoria de trechos.

3. Arquitetura de Agentes - Correção SRP

Arquitetura de 5 agentes orquestrados via LangGraph com Core separado:

Agente 1 - Porteiro (Ingestor): Valida mime-type (pdf/png/jpg), tamanho <20MB, sanitização LGPD. NÃO faz OCR.

Agente 2 - OCR (Escavador OCR): Azure Document Intelligence como primário (96% acurácia em PDFs escaneados jurídicos) com fallback pypdf para PDFs nativos. Retorna texto + bbox + número de páginas.

Agente 3a - Escavador (Extractor): LLM GPT-4o-mini com Structured Output + grounding obrigatório. Prompt exige para cada campo: valor_normalizado, trecho comprobatório verbatim 20-200 chars, pagina_origem, confianca 0-1. Se campo não existir, retorna null.

Agente 3b - Cartógrafo (Normalizer): Validação Pydantic v2 com field_validator que verifica se trecho comprobatório existe no texto OCR (anti-alucinação). Enums fechados: TipoRetroativa (ILIMITADA, DATA_ESPECIFICA, INICIO_VIGENCIA) e CodigoExclusao (DOLO, FRAUDE,

GANHO ILÍCITO, INSURED_VS_INSURED, POLUIÇÃO, MULTAS_PENALIDADES).

Agente 4 - Auditor (Comparator): Compara ApoliceDO campo a campo. Detecta GAP_COBERTURA, GAP_EXCLUSAO e DIVERGÊNCIA. Calcula Score Exposição Oculta: $(GAP_COB*3 + GAP_EXCL*2 + DIV_LIMITE*2 + DIV_RETRO*3)/20*100$. Consulta RAG SUSEP 541/2016 no ChromaDB para contexto Regulatório.

Agente 5 - Relator (Reporter): Streamlit + Mapa de Calor (pandas Styler) com cores Verde=IGUAL, Amarelo=DIVERGÊNCIA, Vermelho=GAP. Gera parecer técnico PDF com ReportLab e XLSX Comparativo.

Core/database.py separado: SQLite para metadados e auditoria de extrações, ChromaDB para trechos e RAG.

Correção aplicada: antes Agente 4 acumulava armazenamento + RAG + comparação + score. Agora storage está no core, respeitando Single Responsibility.

4. Tecnologias Justificadas

Python 3.11: versão estável exigida I2A2.

Azure Document Intelligence: escolhido vs Tesseract (78% vs 96% em PDFs jurídicos escaneados).

Custo MVP: R\$0,15/apólice. Justificativa: Pypdf falha em escaneados, Tesseract perde formatação tabular.

OpenAI GPT-4o-mini Structured Output: custo baixo, suporta response_format=json_object + function calling para schema. Gemini 2.0 Flash mantido como evolução futura para double-check, não no MVP para simplificar.

LangGraph: orquestração de agentes com estado, memória e condicionais. LangChain apenas para prompts.

SQLite + ChromaDB: SQLite para auditoria (tabela extracoes_audit com campo, valor, trecho, página, confiança). ChromaDB para armazenar trechos verbatim e RAG da Circular SUSEP 541/2016. Removido PostgreSQL do MVP para simplificar - overengineering.

Streamlit: interface demonstrável rápida, exigência I2A2.

Pydantic v2: validação + anti-alucinação com context.

ReportLab: geração de parecer técnico PDF.

5. Fluxo de Execução

1. Usuário faz upload de 2 PDFs em app.py (Streamlit).

2. Porteiro valida mime e tamanho.
3. OCR extrai texto com engine informado (azure-di ou pypdf).
4. Escavador chama LLM com prompt de grounding + schema JSON.
5. Cartógrafo valida com Pydantic e verifica se trecho existe no OCR.
6. Sem confiança <0.6, marca para revisão humana no Streamlit.
7. Dados salvos em SQLite + ChromaDB (trecho para auditoria).
8. Auditor compara e calcula Score.
9. Relator gera mapa de calor e parecer PDF/XLSX.

Tempo médio: 30s OCR + 60s LLM + 5s comparação + 5s relatório = ~100s por par.

Modo Mock: se USE MOCK=true ou sem OPENAI_API_KEY, usa regex + dados de data/exemp

6. Mitigação de Riscos

Risco 1 - Alucinação LLM: Mitigado com grounding obrigatório + validador que busca trecho no OCR + confiança + revisão humana se <0.6. Taxa alvo: 100% grounding válido.

Risco 2 - Variabilidade de formatos D&O: Mitigado com schema único com enums fechados e fallback para OUTROS com texto_original.

Risco 3 - Dados sensíveis/LGPD: Usamos apenas apólices fictícias baseadas em modelos Berkley/Austral, anonimizadas, sem CPF/CNPJ real. Não usamos dados reais de clientes.

Risco 4 - Custo API: GPT-4o-mini custa ~R0,05 por apólice. Com 100 apólices teste = R5. Azure DI R\$0,15. Total MVP

Risco 5 - Prazo 06/10/2026: Priorizamos MVP funcional sobre sofisticação. Removemos PostgreSQL, MoviePy, python-pptx do MVP.

Risco 6 - Ambiguidade 'comparação': Implementamos comparação campo a campo + qualitativa (recomendação textual por diferença).

7. Limitações

- Não valida juridicamente: é apoio à subscrição, não parecer legal.
- Depende da qualidade OCR: PDFs muito degradados podem ter confiança baixa.
- Não cobre endossos e cláusulas particulares complexas fora do schema base (20 campos).
- RAG SUSEP 541 limitado a trechos da circular, não jurisprudência.
- Score Exposição Oculta é heurístico, não atuarial.

8. Métricas de Sucesso

- Acurácia extração >90% em 20 campos críticos (validado com ground truth manual)
- Taxa grounding válido = 100% (0 trechos alucinados)
- Tempo: 2h manual -> 4 min automatizado (97% redução)
- Diferenças detectadas: 7-15 por par de apólices
- Testes: pytest com 2 testes de schema e score (tests/test_comparator.py)

9. Evolução Futura

- Suporte D&O; IPO/M&A; com cláusulas específicas
- Gemini 2.0 Flash como segundo LLM para double-check (consenso)
- Validação jurídica com advogados parceiros
- Integração risco de crédito (Serasa)
- Dashboard de carteira com múltiplas apólices

10. Estrutura do Repositório e Instalação

Estrutura conforme exigência I2A2 com Projeto_Final_Artefatos/ contendo pptx, mp4, relatório, arquitetura e xlsx.

Instalação: `git clone https://github.com/InsurTechLabI2A2/do-lens.git`, `python3.11 -m venv venv`, `pip install -r requirements.txt`, `cp .env.example .env`, `streamlit run app.py`

Entregáveis: Relatório Técnico PDF (este), Código ZIP, DOLens_Projeto_Final.pptx,

DOLens_Projeto_Final.mp4, pasta Projeto_Final_Artefatos com artefatos.

Licença MIT.

11. Correções Necessárias Identificadas e Documentadas

Esta seção documenta as correções que tomamos ciência durante a revisão final, mas sem tempo hábil para executar até a data da entrega.

Identificamos a partir da análise do vídeo InsurMinds_Projeto_Final.mp4 (4:50) e da checklist do edital:

A) Filmagem da execução real exigida pelo edital: O vídeo entregue é um pitch animado de alta qualidade, mas não contém 1 segundo de screen recording com app.py rodando. O edital I2A2 exige filmagem da execução com upload real de PDFs, logs dos 5 agentes e geração de artefatos. Correção necessária: gravar com OBS Studio, conforme roteiro documentado em ROTEIRO_GRAVACAO_COMPLEMENTO.md, contendo: `git clone https://github.com/InsurTechLabI2A2/do-lens.git`, `pip install -r requirements.txt`, `streamlit run app.py`,

upload de apolice_alfa_susep_modelo.pdf e apolice_beta_susep_modelo.pdf (LGPD OK), logs de Porteiro (mime/tamanho), OCR (Azure DI 4 e 5 páginas + fallback pypdf), Escavador com grounding (trecho comprovatorio + pagina_origem + confianca), Cartógrafo validando Pydantic anti-alucinação, Auditor calculando Score Exposição Oculta 65 ALTA e Mapa de Calor, Relator gerando XLSX e parecer PDF, SQLite extrações audit e pytest. Tempo estimado: 3min. Documentamos o roteiro, mas não gravamos.

B) Narração no MP4 complementar: O vídeo final gerado DOLens_Projeto_Final_5min_FINAL.mp4 (4:25) foi criado como demonstração simulada via código, sem narração humana. Correção necessária: inserir narração em português explicando cada etapa, com ênfase em 'grounding obrigatório', 'validação Pydantic anti-alucinação', 'Core separado SRP' e 'Score definido'.

C) Edição final única de 5min: O edital pede 1 arquivo MP4 até 5min com pitch + execução. Atualmente temos 2 arquivos: pitch de 4:50 + demo simulada de 4:25. Correção necessária: editar versão final única de 5:00 contendo 0:00-2:00 problema + arquitetura + 2:00-5:00 execução real, com áudio limpo e legendas .SRT sincronizadas. Documentamos SRT, mas não fizemos merge final com OBS.

D) Validação de 7 fluxos completos: No vídeo afirmamos '7 fluxos validados', mas não filmamos os 7. Correção necessária: rodar app com 7 pares diferentes de apólices fictícias e registrar logs.

E) Testes adicionais: Temos apenas 2 testes em tests/test_comparator.py. Correção necessária: adicionar testes de validador de grounding (trecho existe no OCR) e de cálculo de Score.

F) Melhorias de acessibilidade: Legendas .SRT foram geradas (DOLens_Projeto_Final_5min_FINAL.srt) mas não foram embutidas (burn-in) no MP4.